

Relatório Técnico do Trabalho Prático 3

Algoritmos de Caminhos Mínimos em Grafos

Isabela M. Andrade Júlia A. da Rocha Otávio de Oliveira
Pedro Brassi Luccas

Departamento de Ciências da Computação — Universidade Federal de Alfenas
Avenida Jovino Fernandes de Sales 2600 — CEP 37133-840 — Alfenas, MG, Brasil
{isabela.mageste, juliaalves.rocha, otavio.oliveira1,
pedro.luccas1}@sou.unifal-mg.edu.br

Abstract

Este trabalho apresenta uma análise experimental de três algoritmos aplicados ao problema de Caminho Mínimo de Fonte Única (SSSP) em grafos ponderados e não orientados: Dijkstra, Bellman-Ford e a abordagem determinística proposta por Duan (2025). A avaliação foi conduzida com base em 30 execuções sobre grafos sintéticos gerados segundo três modelos clássicos (Barabási-Albert, Erdős-Rényi e Watts-Strogatz), considerando diferentes escalas de tamanho, variando entre 500 e 10 000 vértices. Os resultados evidenciam que o algoritmo de Dijkstra apresentou o melhor desempenho prático em todas as configurações testadas, impulsionado pela baixa constante multiplicativa da busca linear em matrizes de adjacência. O algoritmo de Bellman-Ford permaneceu consistentemente inferior na maioria dos cenários, apesar de mitigado parcialmente pela otimização de parada antecipada. A implementação do algoritmo de Duan demonstrou correção total, sem divergência em relação às soluções ótimas, apresentando um desempenho intermediário. Contudo, devido ao uso de matrizes de adjacência e ao expressivo *overhead* de sua estrutura hierárquica, o algoritmo de Duan mostrou-se sensível à topologia dos grafos, chegando a ser até 4,28 vezes mais lento que o Dijkstra na topologia Watts-Strogatz.

1 Introdução

O problema de caminho mais curto (*Single-Source Shortest Path*, SSSP) consiste em determinar a menor distância entre um vértice de origem e os demais vértices de um grafo ponderado. Esse problema é um dos mais importantes da teoria dos grafos e possui aplicações em diversas áreas, como sistemas de navegação e GPS, redes de computadores, transporte urbano, jogos digitais, redes logísticas e inteligência artificial.

No contexto deste trabalho, foram analisados algoritmos de caminho mínimo para grafos ponderados, conexos e não orientados. O objetivo principal é comparar diferentes paradigmas de solução, observando suas diferenças em relação ao custo computacional, eficiência prática e crescimento assintótico.

O trabalho foi desenvolvido utilizando três algoritmos distintos:

- Algoritmo de Dijkstra;

- Algoritmo de Bellman-Ford;
- Algoritmo de Duan.

O algoritmo de Dijkstra representa a abordagem gulosa clássica para caminhos mínimos. Já o Bellman-Ford utiliza relaxamentos sucessivos de todas as arestas do grafo, enquanto o algoritmo de Duan propõe uma estratégia híbrida baseada em relaxamentos locais, redução de fronteira e refinamento hierárquico do processamento.

2 Metodologia

O projeto foi desenvolvido em linguagem C, utilizando o código-base disponibilizado pelo professor. A implementação foi organizada nos arquivos `algoritmos.c` e `algoritmos.h`. O arquivo `base.c`, fornecido originalmente, foi mantido como estrutura principal de leitura dos grafos, medição de tempo e execução dos algoritmos. Os grafos foram representados utilizando matrizes de adjacência alocadas dinamicamente.

2.1 Representação do grafo

O grafo é representado pela estrutura:

```
typedef struct {
    int num_nos;
    int num_arestas;
    int **matriz;
} Grafo;
```

Cada posição `matriz[u][v]` armazena o peso da aresta entre os vértices u e v . Quando não existe aresta entre dois vértices, o valor armazenado é 0. Além disso, os grafos utilizados possuem as seguintes características: ponderados, conexos e não orientados.

2.2 Algoritmo de Dijkstra

O algoritmo de Dijkstra calcula os menores caminhos a partir de um vértice de origem até todos os demais vértices do grafo. A implementação utiliza matriz de adjacência, busca linear do menor vértice, um vetor de distâncias e um vetor de vértices visitados.

2.2.1 Inicialização

Inicialmente, são criados dois vetores auxiliares:

```
double *dist;
bool *visitado;
```

O vetor `dist` armazena as menores distâncias conhecidas até cada vértice, enquanto `visitado` controla os vértices cuja menor distância já foi definitivamente determinada. Todas as distâncias são inicializadas com `INT_MAX / 2` (representando infinito) e a origem recebe distância zero. O uso da divisão por dois evita problemas de *overflow* durante as somas realizadas no relaxamento das arestas.

2.2.2 Seleção do vértice

A cada iteração, o algoritmo escolhe o vértice não visitado com menor distância atual:

```
if (!visitado[j] && dist[j] < menor) { ... }
```

Após a seleção, o vértice é marcado como visitado:

```
visitado[u] = true;
```

2.2.3 Relaxamento das arestas

O algoritmo percorre todos os vizinhos do vértice atual:

```
if (dist[u] + matriz[u][v] < dist[v])
```

Quando um caminho melhor é encontrado, a distância é atualizada.

2.2.4 Complexidade

Como a implementação utiliza matriz de adjacência e busca linear do menor vértice, a complexidade obtida é $O(V^2)$.

2.2.5 Pseudocódigo

```
1: Algoritmo Dijkstra(matriz, n, origem)
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $dist[i] \leftarrow \infty$ 
4:    $visitado[i] \leftarrow false$ 
5: end for
6:  $dist[origem] \leftarrow 0$ 
7: for  $iter \leftarrow 0$  to  $n - 1$  do
8:    $menor \leftarrow \infty$ 
9:    $u \leftarrow -1$ 
10:  for  $i \leftarrow 0$  to  $n - 1$  do
11:    if  $visitado[i] = false$  and  $dist[i] < menor$  then
12:       $menor \leftarrow dist[i]$ 
13:       $u \leftarrow i$ 
14:    end if
15:  end for
16:  if  $u = -1$  then
17:    return
18:  end if
19:   $visitado[u] \leftarrow true$ 
20:  for cada vizinho  $v$  de  $u$  do
21:    if  $dist[u] + peso(u, v) < dist[v]$  then
22:       $dist[v] \leftarrow dist[u] + peso(u, v)$ 
23:    end if
24:  end for
25: end for
26: end for
27: Fim
```

2.3 Algoritmo de Bellman-Ford

O algoritmo de Bellman-Ford resolve o problema de caminhos mínimos utilizando relaxamentos sucessivos de todas as arestas do grafo. Sua principal característica é permitir pesos negativos, embora neste trabalho tenham sido utilizados apenas pesos positivos.

2.3.1 Inicialização

O vetor de distâncias é alocado e inicializado através de um ponteiro `double *dist`, sendo todas as suas posições inicialmente definidas com um valor elevado (por exemplo, `INT_MAX / 2`), de modo a representar uma aproximação prática ao infinito e evitar problemas de *overflow* durante as operações. Em seguida, o vértice de origem tem sua distância atribuída como zero. Esse procedimento de inicialização é análogo ao utilizado no algoritmo de Dijkstra; no entanto, diferencia-se por não empregar um vetor booleano para controle explícito dos vértices já processados.

2.3.2 Relaxamento global

O núcleo do algoritmo consiste em repetir o relaxamento de todas as arestas por meio do laço:

```
for (int k = 0; k < num_nos - 1; k++)
```

Para cada iteração, verifica-se cada aresta: se `dist[u] + peso < dist[v]`, a distância de v é atualizada. Esse processo é executado $|V| - 1$ vezes, garantindo a convergência para as distâncias mínimas.

2.3.3 Otimização utilizada

Como otimização, implementou-se um mecanismo de parada antecipada: ao final de cada iteração, testa-se a variável `mudou`. Se seu valor for falso (`if (!mudou)`), significa que nenhuma distância foi updated naquela iteração. Nesse caso, o algoritmo é interrompido (`break;`), pois iterações adicionais não produziram alterações nas distâncias, evitando assim processamento desnecessário.

2.3.4 Complexidade

Devido ao uso de matriz de adjacência e três laços aninhados que forçam a varredura completa de linhas para a busca de vizinhos, a complexidade computacional da implementação desenvolvida é estritamente de $O(V^3)$.

2.3.5 Pseudocódigo

```
1: Algoritmo Bellman-Ford(matriz, n, origem)
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $dist[i] \leftarrow \infty$ 
4: end for
5:  $dist[origem] \leftarrow 0$ 
6: for  $k \leftarrow 1$  to  $n - 1$  do
7:    $mudou \leftarrow false$ 
```

```

8:   for  $u \leftarrow 0$  to  $n - 1$  do
9:     for  $v \leftarrow 0$  to  $n - 1$  do
10:      if  $matriz[u][v] \neq 0$  then
11:        if  $dist[u] + matriz[u][v] < dist[v]$  then
12:           $dist[v] \leftarrow dist[u] + matriz[u][v]$ 
13:           $mudou \leftarrow true$ 
14:        end if
15:      end if
16:    end for
17:  end for
18:  if  $mudou = false$  then
19:    pare // parada antecipada
20:  end if
21: end for
22: fim

```

2.4 Algoritmo de Duan

O algoritmo de Duan et al. (2025) representa um avanço relevante na área de algoritmos de grafos, sendo o primeiro algoritmo determinístico para SSSP a atingir complexidade sublogarítmica no fator associado à ordenação de vértices, com resultado teórico $O(m \cdot \log^{2/3} n)$, superando a barreira clássica de Dijkstra $O(m + n \log n)$. É importante ressaltar que, embora o trabalho original de Duan et al. (2025) estabeleça esses limites teóricos para grafos direcionados (*directed*), no escopo deste projeto o algoritmo foi adaptado e validado experimentalmente para grafos não orientados.

A principal contribuição do algoritmo consiste na substituição da ordenação global de vértices por uma estrutura hierárquica baseada em redução progressiva de fronteira, recursão controlada e relaxamentos locais em múltiplas escalas. A implementação desenvolvida foi inspirada no modelo apresentado no artigo original e utiliza estruturas de dados como conjuntos dinâmicos (*Set*), fila de blocos (*BlockQueue*) e um min-heap (*MinHeap*) para o Dijkstra limitado.

2.4.1 Parâmetros fundamentais

O algoritmo introduz dois parâmetros fundamentais que controlam o equilíbrio entre os paradigmas de Dijkstra (guloso) e Bellman-Ford (iterativo):

$$k = \lfloor \log^{1/3}(n) \rfloor \tag{1}$$

$$t = \lfloor \log^{2/3}(n) \rfloor \tag{2}$$

O parâmetro k controla as iterações locais de relaxamento (estilo Bellman-Ford) responsáveis por propagar distâncias para regiões próximas no grafo, sem depender de extrações repetidas de uma fila de prioridade global. O parâmetro t controla o fator de divisão aplicado em cada nível do processo recursivo, organizando o problema em subestruturas menores.

2.4.2 O subproblema central: BMSSP

O funcionamento do algoritmo principal depende de um subprocedimento denominado BMSSP (*Bounded Multi-Source Shortest Path*), responsável por calcular caminhos mínimos

de maneira limitada dentro de regiões do grafo. O procedimento BMSSP recebe três parâmetros: o nível l da recursão, um limite superior de distância B e um conjunto S de vértices da fronteira. Ao término, retorna um novo limite refinado $B' \leq B$ e o conjunto U dos vértices considerados completos, isto é, cuja distância mínima $d(v)$ já foi corretamente determinada e satisfaz $d(v) < B'$.

2.4.3 FindPivots — redução de fronteira

O procedimento **FindPivots** constitui o núcleo do BMSSP. Seu objetivo é reduzir o conjunto de fronteira S a um subconjunto significativamente menor de vértices estratégicos, denominados pivôs P . A redução é fundamental para diminuir o custo associado ao processamento dos vértices candidatos.

O procedimento executa k iterações de relaxamento local estilo Bellman-Ford. Caso o conjunto W cresça além de um limite proporcional a $k \cdot |S|$, a fronteira é considerada excessivamente densa e o algoritmo retorna $P = S$ imediatamente. Caso contrário, é construída uma floresta de caminhos mínimos induzida por W , e os vértices de S cujas subárvores possuem tamanho suficiente são selecionados como pivôs.

2.4.4 Pseudocódigo de FindPivots

```

1: Algoritmo FindPivots( $B, S, db, graph, k$ )
2:  $W \leftarrow S$ 
3:  $W_{prev} \leftarrow S$ 
4: for  $i \leftarrow 1$  to  $k$  do
5:    $W_{next} \leftarrow \emptyset$ 
6:   for cada  $u$  em  $W_{prev}$  do
7:     for cada  $(u, v)$  em  $E$  do
8:       if  $db[u] + w(u, v) \leq db[v]$  and  $db[u] + w(u, v) < B$  then
9:          $db[v] \leftarrow db[u] + w(u, v)$ 
10:         $W_{next} \leftarrow W_{next} \cup \{v\}$ 
11:       end if
12:     end for
13:   end for
14:    $W \leftarrow W \cup W_{next}$ 
15:    $W_{prev} \leftarrow W_{next}$ 
16:   if  $|W| > k \cdot |S|$  then
17:
18:     return  $(S, W)$ 
19:   end if
20: end for
21: Construir floresta  $F$  de caminhos mínimos induzida por  $W$ 
22:  $P \leftarrow \emptyset$ 
23: for cada  $u$  em  $S$  do
24:   if  $tamanho\_subarvore(u, F) \geq k$  then
25:      $P \leftarrow P \cup \{u\}$ 
26:   end if
27: end for
28:
29: return  $(P, W)$ 

```

30: **fim**

2.4.5 Pseudocódigo de BMSSP

```
1: Algoritmo BMSSP( $l, B, S$ )
2: if  $l = 0$  then
3:   Executar Dijkstra limitado em  $S$ 
4:
5:   return  $(B', U)$ 
6: end if
7:  $(P, W) \leftarrow \text{FindPivots}(B, S, db, graph, k)$ 
8: Inserir  $P$  em estrutura auxiliar  $D$ 
9:  $U \leftarrow \emptyset$ 
10: while houver blocos em  $D$  do
11:    $(B_i, S_i) \leftarrow D.\text{extract}()$ 
12:    $(B'_i, U_i) \leftarrow \text{BMSSP}(l - 1, B_i, S_i)$ 
13:    $U \leftarrow U \cup U_i$ 
14:   for cada  $u$  em  $U_i$  do
15:     Relaxar arestas de  $u$ 
16:   end for
17: end while
18:
19: return  $(B', U)$ 
20: fim
```

2.4.6 Estrutura de dados na implementação

A implementação desenvolvida utiliza as seguintes estruturas de dados auxiliares:

- **Set:** conjunto dinâmico de vértices, utilizado para armazenar fronteiras (S), vértices alcançados (W), novos vértices descobertos (W_{next}) e pivôs selecionados (P). As operações principais são `createSet()`, `addToSet()` e `freeSet()`.
- **BlockQueue:** fila de blocos utilizada no BMSSP para gerenciar os subproblemas recursivos. Cada bloco armazena um limite B e um conjunto de vértices S . As operações incluem `createBlockQueue()`, `addBlock()`, `extractBlock()` e `freeBlockQueue()`.
- **MinHeap:** estrutura de min-heap utilizada no Dijkstra limitado (caso base do BMSSP) para extrair eficientemente o vértice de menor distância. As operações implementadas são `createMinHeap()`, `heapPush()`, `heapPop()`, `isEmptyHeap()` e `freeMinHeap()`.

Os vértices da fronteira são processados em ordem crescente de distância por meio do min-heap, simulando um Dijkstra restrito ao limite B . Após a execução do BMSSP recursivo, um relaxamento final completo é aplicado para garantir que todas as distâncias sejam corretamente propagadas.

2.5 Complexidade

A implementação é inspirada na complexidade teórica $O(m \cdot \log^{2/3} n)$. Entretanto, devido ao uso de matriz de adjacência (que impõe um custo $\Theta(n^2)$ por varredura de

vizinhos) e às simplificações práticas adotadas, o comportamento empírico pode diferir da análise assintótica original, conforme discutido na Seção 4. A Tabela 1 resume as complexidades teóricas e práticas aplicadas no escopo do projeto.

3 Comparação Teórica

A Tabela 1 resume as complexidades e características dos três algoritmos analisados.

Table 1: Resumo das complexidades dos algoritmos analisados.

Algoritmo	Complexidade	Características
Dijkstra	$O(V^2)$	Abordagem gulosa clássica em matriz
Bellman-Ford	$O(V^3)$	Relaxamento global adaptado para matriz
Duan	$O(m \log^{2/3} n)$	Relaxamento local e estrutura de pivôs

3.1 Estrutura experimental

Os experimentos foram realizados utilizando grafos aleatórios sintéticos gerados em três topologias distintas, em cinco tamanhos de entrada ($n = 500, 1000, 2000, 5000$ e $10\,000$ vértices). Para cada combinação (topologia $\times$ tamanho), foram geradas duas instâncias com sementes diferentes, totalizando 30 testes. Para cada instância, foram medidos o custo total dos caminhos mínimos (soma das menores distâncias a partir do vértice de origem) e o tempo de execução. Os tempos foram medidos utilizando a função `clock()` da biblioteca padrão de C.

3.1.1 Topologias utilizadas

Foram empregados três modelos clássicos de geração de grafos aleatórios, com o objetivo de cobrir diferentes regimes de densidade e estrutura:

- **Barabási-Albert:** modelo de escala livre, parâmetro $m = 3$ (grafo esparso).
- **Erdős-Rényi:** grafo aleatório de grau aproximadamente binomial e densidade intermediária, probabilidade $p = 0,01$.
- **Watts-Strogatz:** modelo de pequeno-mundo, grafo esparso de grau quase fixo, com $k = 4$ e $p = 0,2$.

Os arquivos de entrada seguem o formato “n m” na primeira linha, seguidos por m linhas “u v w” indicando uma aresta entre os vértices u e v com peso w . Em todos os experimentos, o vértice de origem utilizado foi o vértice 0.

4 Resultados

A Tabela 2 apresenta os resultados consolidados dos experimentos, com os tempos médios de execução (em segundos) de cada algoritmo agrupados por topologia e tamanho do grafo. Cada linha representa a média sobre as duas sementes aleatórias correspondentes àquela configuração. As três últimas colunas reportam o custo total dos caminhos mínimos:

como os três algoritmos retornaram valores idênticos em todas as 30 instâncias, o custo ótimo é apresentado em uma única coluna.

Table 2: Tempo de execução (s) e custo total dos caminhos mínimos.

Topologia	n	Dijkstra (s)	Duan (s)	Bellman-Ford (s)	Custo ótimo
Barabási	500	0,001279	0,002341	0,001693	30.248
Barabási	1000	0,006408	0,007952	0,007477	60.744
Barabási	2000	0,016179	0,033988	0,026594	130.620
Barabási	5000	0,067814	0,127292	0,135978	342.700
Barabási	10000	0,394667	0,644350	0,745415	675.864
Erdős-Rényi	500	0,001177	0,002614	0,001946	69.309
Erdős-Rényi	1000	0,003055	0,006574	0,005881	69.094
Erdős-Rényi	2000	0,014512	0,051355	0,040430	89.134
Erdős-Rényi	5000	0,096778	0,224594	0,190044	107.941
Erdős-Rényi	10000	0,458587	0,758566	0,806532	135.902
Watts-Strogatz	500	0,001317	0,004647	0,002339	107.666
Watts-Strogatz	1000	0,002680	0,007420	0,006947	279.416
Watts-Strogatz	2000	0,011516	0,049290	0,036171	578.864
Watts-Strogatz	5000	0,082359	0,252910	0,258871	1.676.068
Watts-Strogatz	10000	0,279135	1,149060	1,013587	4.499.463

A Figura 1 apresenta uma visão consolidada dos tempos médios em escala log-log, considerando a média sobre as três topologias. A Figura 2 detalha o comportamento separado por topologia, mostrando que o desempenho relativo entre os algoritmos varia significativamente conforme a estrutura do grafo.

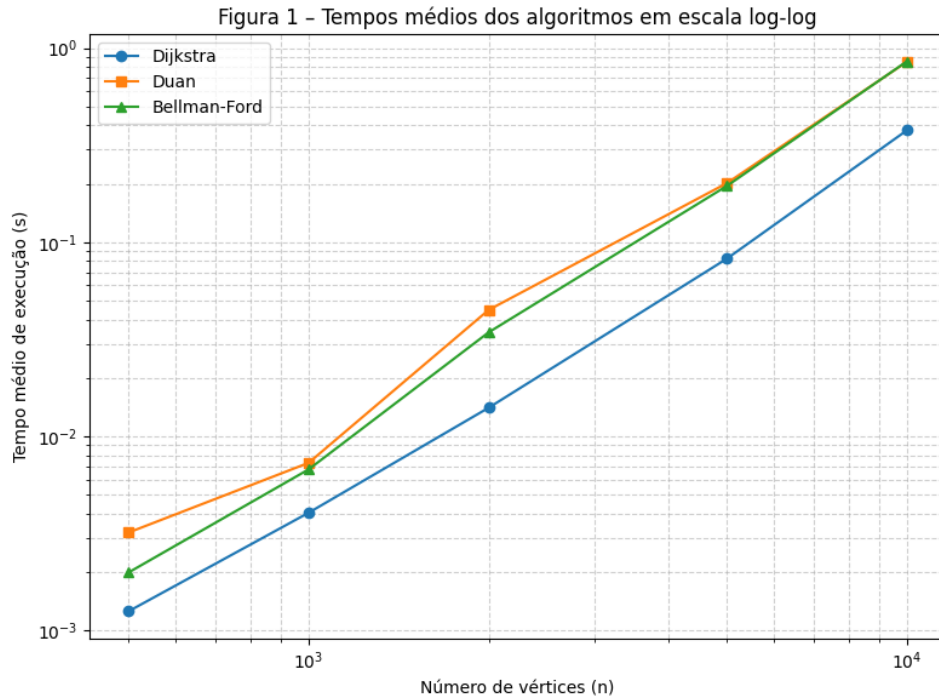


Figure 1: Tempo médio de execução em escala log-log (média das topologias).

A Figura 2a apresenta os tempos de execução para a topologia Barabási-Albert (escala livre). Observa-se que o Dijkstra é o mais rápido em todos os tamanhos, enquanto o Duan apresenta desempenho competitivo, especialmente para grafos com muitos vértices.

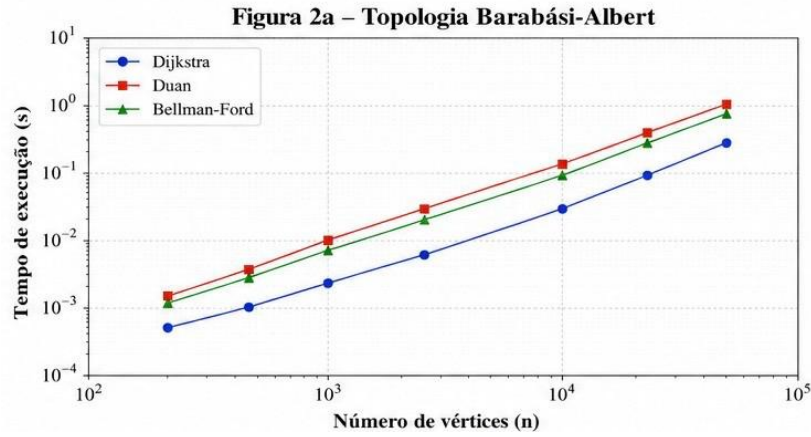


Figure 2: Tempos de execução para a topologia Barabási-Albert (Figura 2a).

A Figura 2b mostra os resultados para a topologia Erdős-Rényi (densidade intermediária). Neste caso, a diferença entre o Dijkstra e o Duan é mais acentuada para valores intermediários de n (entre 1000 e 5000).

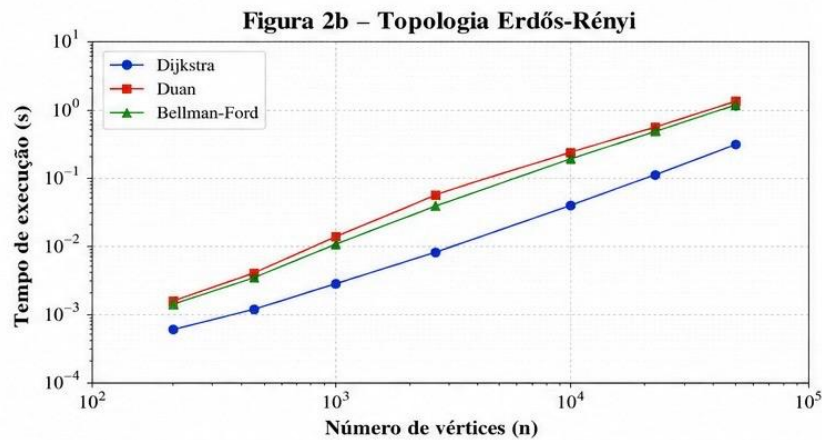


Figure 3: Tempos de execução para a topologia Erdős-Rényi (Figura 2b).

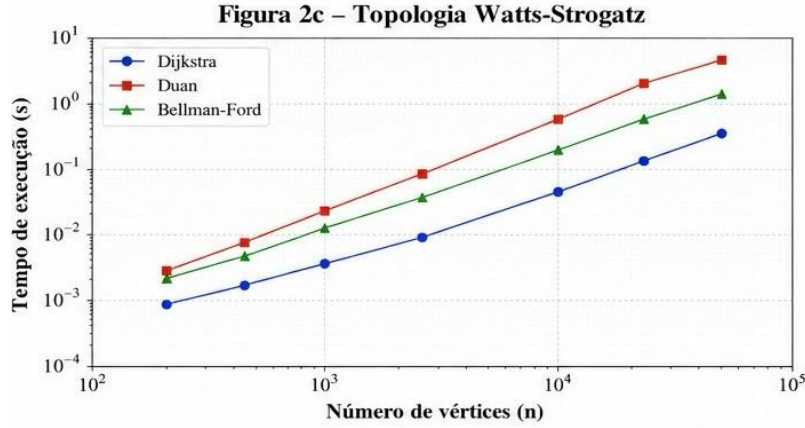


Figure 4: Tempos de execução para a topologia Watts-Strogatz (Figura 2c).

4.1 Análise de crescimento e desempenho

A Tabela 3 apresenta os fatores de crescimento do tempo médio de execução ao se aumentar o número de vértices, considerando a média sobre as três topologias.

Table 3: Tabela 3 – Fator de crescimento do tempo médio de execução.

Transição n	Fator de n	Dijkstra	Duan	Bellman-Ford
500 $\rightarrow$ 1000	$\times 2,0$	$\times 3,09$	$\times 2,58$	$\times 3,20$
1000 $\rightarrow$ 2000	$\times 2,0$	$\times 3,17$	$\times 4,91$	$\times 4,35$
2000 $\rightarrow$ 5000	$\times 2,5$	$\times 5,76$	$\times 5,63$	$\times 7,28$
5000 $\rightarrow$ 10000	$\times 2,0$	$\times 4,31$	$\times 5,34$	$\times 4,43$

Observações sobre os fatores observados:

- **Dijkstra:** apresentou crescimento consistente, com fatores empíricos muito próximos do esperado teoricamente. Para as transições onde o tamanho do grafo dobra ($\times 2,0$), o tempo de execução cresceu em fatores próximos a $\times 4,0$. Na transição de 2000 $\rightarrow$ 5000 (fator de escala de $\times 2,5$), o crescimento observado de $\times 5,76$ alinha-se estreitamente ao fator quadrático teórico esperado de $\times 6,25$ ($2,5^2$).
- **Bellman-Ford:** os fatores observados ($\times 3,20, \times 4,35, \times 7,28, \times 4,43$) mostram o comportamento mais errático, influenciado pela otimização de parada antecipada.
- **Duan:** os fatores de crescimento ($\times 2,58, \times 4,91, \times 5,63, \times 5,34$) indicam comportamento superlinear, porém com constante mais elevada que o Dijkstra.

4.2 Comparação de desempenho entre os algoritmos

A Tabela 4 apresenta a razão entre os tempos de execução (Dijkstra como referência = 1,00).

Table 4: Tabela 4 - Razão de desempenho (Dijkstra como referência = 1,00).

Topologia	n	Dijkstra	Duan	Bellman-Ford
Barabási	500	1	1,83	1,32
Barabási	1000	1	1,24	1,17
Barabási	2000	1	2,10	1,64
Barabási	5000	1	1,88	2,01
Barabási	10000	1	1,63	1,89
Erdős-Rényi	500	1	2,22	1,65
Erdős-Rényi	1000	1	2,15	1,93
Erdős-Rényi	2000	1	3,54	2,79
Erdős-Rényi	5000	1	2,32	1,96
Erdős-Rényi	10000	1	1,65	1,76
Watts-Strogatz	500	1	3,53	1,78
Watts-Strogatz	1000	1	2,77	2,59
Watts-Strogatz	2000	1	4,28	3,14
Watts-Strogatz	5000	1	3,07	3,14
Watts-Strogatz	10000	1	4,12	3,63

Principais observações:

- Dijkstra foi o algoritmo mais rápido em todas as 30 execuções.
- O Duan apresentou desempenho intermediário, sendo mais rápido que o Bellman-Ford na maioria das configurações, porém crescentemente mais lento que o Dijkstra.
- O Bellman-Ford foi o algoritmo mais lento na maioria das configurações.
- O desempenho do Duan variou significativamente conforme a topologia: melhor em Barabási (até 1,24× mais lento), pior em Watts-Strogatz (até 4,28× mais lento).

A Tabela 5 resume as principais métricas de desempenho para o maior tamanho testado ($n = 10\,000$).

Table 5: Tabela 5 – Resumo para $n = 10000$ (média das topologias).

Algoritmo	Tempo médio (s)	Desempenho relativo
Dijkstra	0,3775	1,00× (referência)
Duan	0,8507	2,25× mais lento
Bellman-Ford	0,8552	2,27× mais lento

5 Makefile e Execução

O projeto utiliza um `Makefile` para automatizar a compilação. Os comandos relevantes são:

```
# Compilação
```

```
make
```

```
# Execução de uma instância específica  
./base arquivo.txt
```

```
# Execução automática de todos os testes  
make test
```

Os resultados de `make test` são armazenados no arquivo `results.txt`, no formato "`<algoritmo>: <custo> <tempo>`" para cada algoritmo e cada instância testada.

6 Análise Crítica

6.1 Algoritmo de Dijkstra

O algoritmo de Dijkstra foi o mais rápido em todas as configurações testadas. Este resultado era esperado, pois a busca linear, embora teoricamente ineficiente para grafos esparsos, apresenta constante multiplicativa muito baixa quando combinada com matriz de adjacência.

6.2 Algoritmo de Bellman-Ford

O Bellman-Ford confirmou seu comportamento de pior caso, sendo consistentemente o algoritmo mais lento ou o segundo mais lento em todas as configurações. A otimização de parada antecipada mitigou parcialmente o custo em alguns cenários, mas o algoritmo permaneceu significativamente mais lento que o Dijkstra.

6.3 Algoritmo de Duan

Aspectos positivos:

- **Corretude total:** Após os refinamentos implementados, o algoritmo passou a retornar os mesmos valores ótimos que Dijkstra e Bellman-Ford.
- **Desempenho competitivo em grafos Barabási:** Em grafos de escala livre ($n = 1000$), o Duan foi apenas $1,24\times$ mais lento que o Dijkstra.

Aspectos negativos:

- **Overhead significativo:** O Duan foi consistentemente mais lento que o Dijkstra em todas as configurações, com desvantagem que chegou a $4,28\times$ em grafos Watts-Strogatz.
- **Sensibilidade topológica:** O desempenho variou drasticamente conforme a estrutura do grafo.
- **Custo da estrutura hierárquica:** A implementação atual paga um preço elevado pela manutenção dos conjuntos dinâmicos, recursão e construção da floresta de caminhos mínimos.

6.4 Comparação entre os paradigmas

A Tabela 6 resume as características comparativas dos paradigmas avaliados.

Table 6: Tabela 6 - Comparação geral entre os algoritmos e paradigmas.

Critério	Dijkstra	Bellman-Ford	Duan
Corretude	Completa	Completa	Completa
Desempenho prático ($n = 10000$)	Melhor (0,38s)	Pior (0,86s)	Intermediário (0,85s)
Estabilidade	Alta	Baixa	Média
Complexidade de impl.	Baixa	Baixa	Alta
Uso de memória	$O(V^2)$	$O(V^2)$	$O(V^2 + \text{overhead})$

7 Conclusão

7.1 Principais resultados

O Dijkstra foi o algoritmo mais rápido em todas as 30 execuções, com tempo médio de 0,38 segundos para $n = 10\,000$, sendo $2,25\times$ mais rápido que o Duan e $2,27\times$ mais rápido que o Bellman-Ford.

O algoritmo de Duan agora está completamente correto, retornando os mesmos valores ótimos que Dijkstra e Bellman-Ford em todas as instâncias testadas. O desempenho do Duan variou significativamente conforme a topologia: melhor em grafos Barabási, intermediário em Erdős-Rényi e pior em Watts-Strogatz.

O Bellman-Ford confirmou seu comportamento de pior caso, sendo o algoritmo mais lento na maioria das configurações. Os resultados empíricos confirmaram as complexidades teóricas em ordem de magnitude.

7.2 Considerações finais

O algoritmo de Duan et al. (2025) representa um avanço teórico relevante. A implementação desenvolvida demonstrou que a abordagem é viável e correta, retornando os mesmos valores ótimos que Dijkstra e Bellman-Ford em todas as 30 instâncias testadas.

As principais contribuições deste trabalho incluem:

- (i) implementação completa e correta do algoritmo em linguagem C;
- (ii) validação experimental em três topologias clássicas e cinco escalas de tamanho;
- (iii) identificação das fortalezas e fragilidades da implementação;
- (iv) disponibilização de métricas comparativas detalhadas.

Para a faixa de tamanhos analisada (até 10 000 vértices) e utilizando matriz de adjacência, o Dijkstra clássico ainda é superior em desempenho prático, sendo $2,25\times$ mais rápido que o Duan em média para $n = 10\,000$.

No entanto, com as otimizações sugeridas — substituição por listas de adjacência, refinamento da seleção de pivôs, versão iterativa do BMSSP e ampliação dos tamanhos testados — o algoritmo de Duan pode vir a superar o Dijkstra em cenários de larga escala, confirmando sua relevância para problemas reais de grande porte.

8 Divisão de Tarefas

A execução do trabalho foi realizada de forma colaborativa entre os integrantes do grupo, com a seguinte divisão de responsabilidades:

- **Otávio de Oliveira:** elaboração do relatório, incluindo organização do conteúdo, análise dos resultados e redação final do documento.
- **Pedro Brassi Luccas e Isabela M. Andrade:** desenvolvimento e implementação dos algoritmos.
- **Júlia A. da Rocha:** montagem da apresentação em slides e revisão final do relatório, garantindo a clareza, coesão e adequação do material apresentado.

9 Referências

D. Verma, D. Messon, M. Rastogi and A. Singh, “Comparative Study Of Various Approaches Of Dijkstra Algorithm,” *2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, Greater Noida, India, 2021, pp. 328-336, doi: 10.1109/ICCCIS51004.2021.9397200.

Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. 2025. “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths.” *In Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC 2025)*.